MATERNAL DIGITAL TWIN · PREECLAMPSIA EARLY WARNING

PregTwin (Health-Twin)

A longitudinal maternal digital twin and preeclampsia early-warning research platform: README, architecture and full codebase walkthrough

Vatsal Kumar · Team kvatsal03
Ramaiah Institute of Technology, Bengaluru · ISE
github.com/vatsalpro/health-twin



Why PregTwin exists: the silent window before preeclampsia

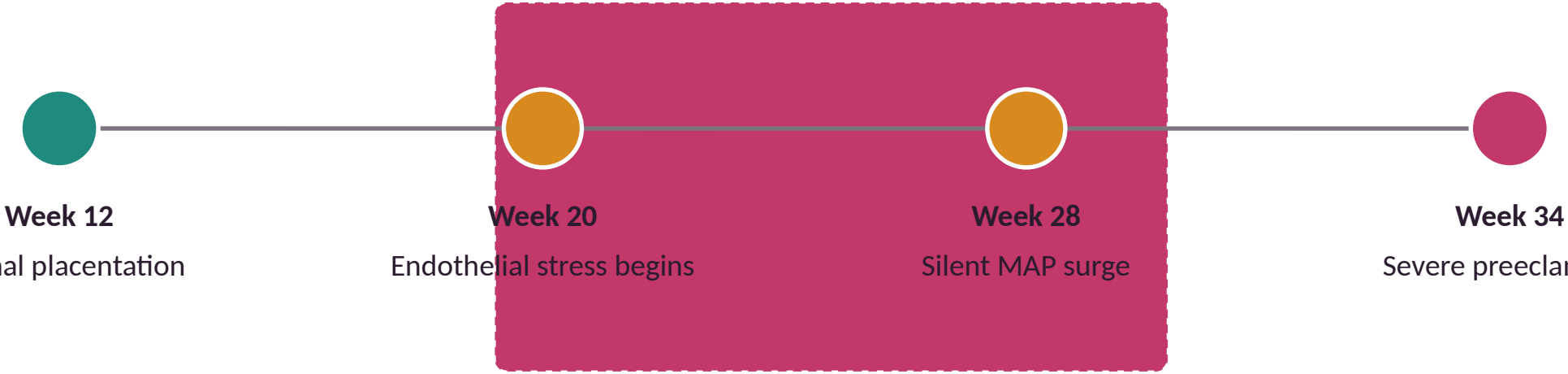
800+
women die every day from preventable pregnancy complications

2 min
one maternal death every two minutes worldwide

14 days
forecast horizon of the early-warning risk model

≥ 20 wk
ACOG / ISSHP gestational-age eligibility guard

How preeclampsia progresses without alarms



Every beat matters. Every mother matters.

Eight core capabilities

Living 3D twin

Three.js + GLSL shaders react to MAP, BP and gestational age

What-if simulation lab

Cardio, renal and metabolic scenarios with 14-day risk deltas

Contactless camera

MediaPipe 478-point mesh + CHROM rPPG pulse, all in-browser

Clinical document OCR

PyMuPDF coordinates + local Ollama; clinician reviews every value

Dr. Crane AI mascot

Patient chat through Requesty; explains orders, never prescribes

Emergency triage

Rule engine checks ACOG/ISSHP red flags before any LLM runs

Medication management

Clinician-set orders, dose reminders and in-app confirmation

Scheduling & booking

Weekly availability builder, atomic slot booking, doctor alerts

System architecture: five layers plus a data layer

1 · Ingestion	2 · Gateway & safety	3 · Dual AI (isolated)	4 · Twin core	5 · Experience
Antenatal PDFs and lab reports	JWT + RBAC: clinician / patient isolation	Local Ollama llama3.2:3b on :11434. Zero egress. OCR and lab parsing	Twin state engine: MAP and hemodynamics	Three.js 3D living twin
IoT vitals: BLE BP, SpO2, scale, CGM	Emergency rule engine (ACOG / ISSHP)	Requesty gateway to Dr. Crane (gemma-4-31b-it). Patient chat only	14-day Platt-calibrated preeclampsia risk	Patient portal + Dr. Crane chat
Webcam: MediaPipe + rPPG	Intent classifier and prompt-injection defence		Counterfactual simulation lab	Clinician workstation + audit trail
Patient messages and symptoms				Welcome screen, origami UI
Clinician orders and schedule				

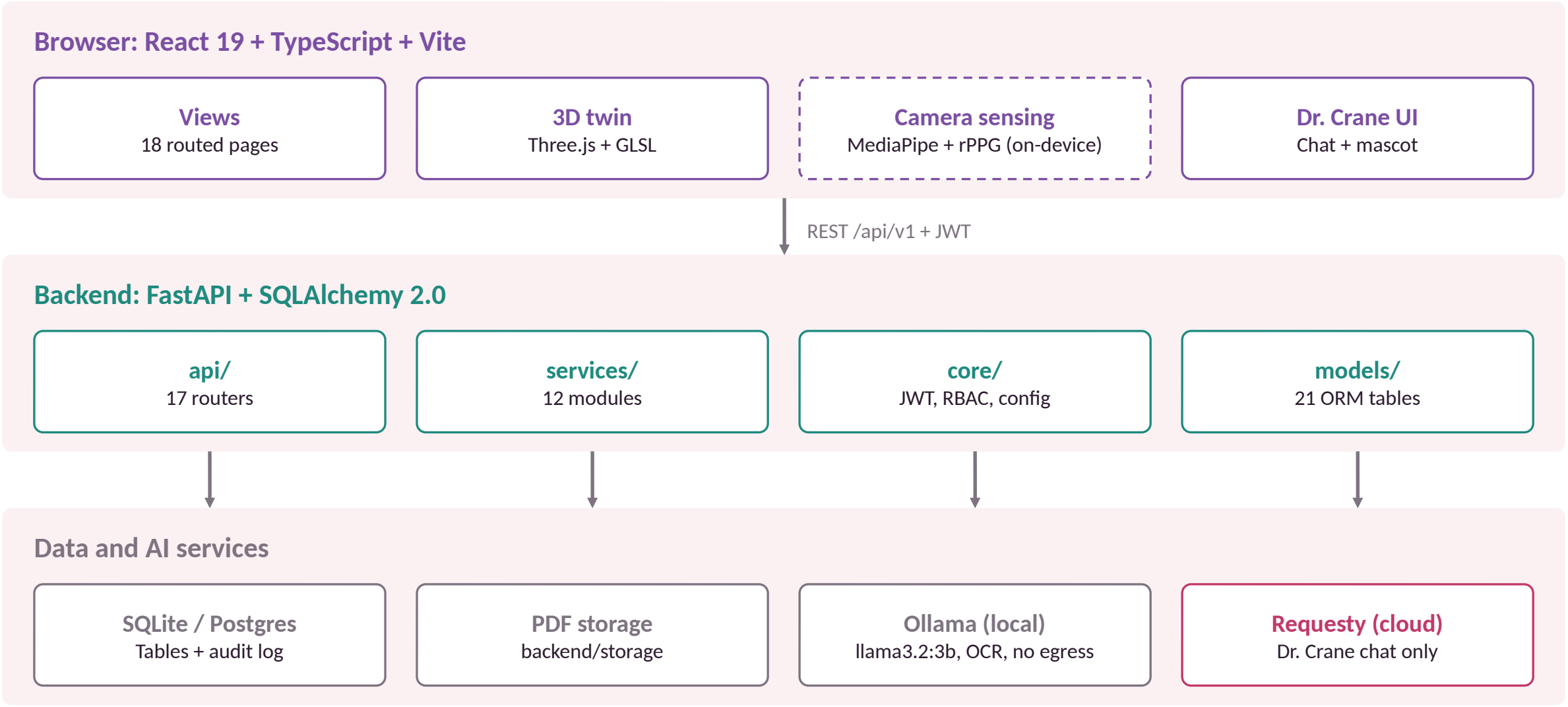
Care coordination engine

Medication engine · schedule builder and atomic booking · emergency escalation to on-call clinician · dose-reminder sync

Persistence & audit

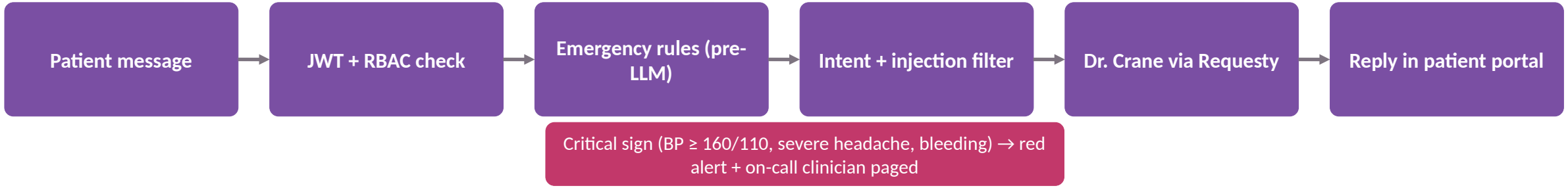
SQLAlchemy 2.0 → SQLite / PostgreSQL · 21 ORM tables · audit events · local PDF storage

Runtime architecture: browser, backend, data and AI

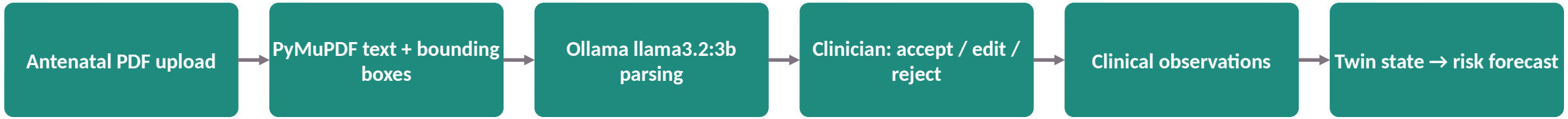


Three request paths through the system

Patient chat path: safety gates run before any AI



Document path: human-in-the-loop extraction



Camera path: 100% on-device, no video stored



Tech stack and strict AI isolation

Backend

- FastAPI + Uvicorn, async REST under /api/v1
- SQLAlchemy 2.0: SQLite, PostgreSQL-ready
- JWT + PBKDF2-SHA256; roles: clinician, patient, admin
- PyMuPDF 2 + Tesseract OCR adapter
- scikit-learn, NumPy, SciPy, httpx
- pytest: 72 tests

Frontend

- React 19 + TypeScript + Vite 8, React Router 7
- Tailwind CSS v4, glassmorphic clinical UI
- Three.js with custom GLSL shaders
- MediaPipe Face Landmarker (WASM) + CHROM rPPG
- Recharts trajectories and contribution charts
- Vitest: 25 tests

Ollama: private, compulsory

llama3.2:3b at localhost:11434 (~2 GB). Clinical OCR and lab parsing with zero data egress.

Requesty: chat only

router.requesty.ai/v1 with google/gemma-4-31b-it. Powers Dr. Crane; architecturally separate from Ollama.

1. Living 3D maternal twin

Turns hemodynamic data into an organic 3D maternal figure, so clinicians see trends instead of reading a spreadsheet.

How it works

1. Twin state engine derives MAP and pulse pressure from every new SBP / DBP observation
2. The resulting MaternalTwinState is stored per patient and served by the twin API
3. Three.js loads maternal-body.glb and runs two custom GLSL shaders
4. Shader B ties pulse speed, ring frequency and particle drift to MAP, BP and gestational age

Key details

- Shader A: ambient cellular field from multi-scale simplex noise (micro-perfusion, oxygenation)
- Shader B: data-reactive hemodynamic signal field
- Shown in the Maternal Twin view for both roles

Where in the code

```
components/MaternalTwin3D.tsx
views/MaternalTwinView.tsx
public/models/maternal-body.glb
services/twin_service.py
api/twin.py
```

2. What-if simulation lab

A risk-free counterfactual playground where clinicians change inputs and see the projected 14-day preeclampsia probability move.

How it works

1. Pick a module: cardiovascular, metabolic, sleep and stress, sensor reliability, monitoring schedule, gestational or postpartum timeline
2. Adjust inputs such as SBP, DBP, heart rate, arterial stiffness, glucose, HbA1c, UPCR, platelets
3. Backend re-runs the risk model on the modified features and returns the delta
4. Save, duplicate, compare and export scenarios as authenticated CSV or JSON

Key details

- 8 simulation functions in simulation_service.py
- Largest screen in the app (2,350 lines)
- Scenarios are labeled non-causal exploratory hypotheses

Where in the code

```
views/SimulationLabView.tsx
api/simulation.py
services/simulation_service.py
services/prediction_service.py
table: SimulationScenario
```

3. Contactless camera and stress sensing

Estimates pulse and facial-tension indicators from an ordinary webcam, entirely inside the browser.

How it works

1. MediaPipe Face Landmarker tracks 478 landmarks and 52 blendshapes, plus head pose and blink rate
2. CHROM rPPG samples forehead and cheek regions, projects color channels, detrends, then finds the pulse peak by DFT (0.75 to 3 Hz)
3. HRV needs 60 s of stable video, otherwise shows INSUFFICIENT_DURATION
4. Only derived numbers are stored, and only with participant consent

Key details

- No video frame is recorded or transmitted
- Stress classifier is honestly marked UNAVAILABLE_AWAITING_VALIDATED_CLASSIFIER
- Runs on bundled WASM + float16 model

Where in the code

```
services/faceLandmarkerService.ts
services/rppgProcessor.ts
services/stressAnalysisPipeline.ts
views/CameraSignalResearchView.tsx
api/camera_research.py
```

4. Clinical document understanding

Reads unstructured antenatal PDFs, extracts vitals and labs with their page coordinates, and keeps a clinician in the loop.

How it works

1. Upload a PDF; PyMuPDF pulls text and bounding boxes for each value
2. Deterministic patterns plus local Ollama (llama3.2:3b) extract BP, urinalysis, UPCR, platelets, AST, ALT, creatinine, gestational age
3. Context-aware dates separate current-visit values from historical baselines
4. Clinician compares each value with the PDF, then accepts, edits or rejects; accepted values become observations

Key details

- Accept-all keeps corrections and never revives rejected fields
- Idempotent: repeat runs create no duplicates
- Every decision is written to the audit log

Where in the code

```
api/documents.py
services/document_processor.py
services/ai_extractor.py
views/DocumentDetailView.tsx
views/ClinicianReviewQueueView.tsx
```

5. Risk forecast and clinical guardrails

Estimates the probability of clinician-confirmed preeclampsia within 14 days, with strict ACOG / ISSHP safety rules.

How it works

1. Gate: gestational age must be 20.0 to 41.0 weeks, otherwise INSUFFICIENT_DATA
2. Features are read only at or before the prediction time (zero temporal leakage)
3. Logistic model (v1.4.2) combines risk factors, BP, MAP, proteinuria, labs and SBP trend
4. Output: Platt-calibrated probability, 95% CI and ranked feature contributions

Key details

- Evidence levels: verified, patient-reported, AI-extracted, clinician-verified, model forecast, counterfactual
- Evaluation reports not_evaluated, never invented AUROC

Where in the code

```
services/prediction_service.py
api/predictions.py
views/RiskForecastView.tsx
table: RiskPrediction
```

6. Dr. Crane AI doctor mascot

A friendly patient-facing crane doctor that explains orders and results, with safety checks before and after the model.

How it works

1. Patient message passes JWT and role checks, then the emergency rule engine
2. Intent classifier and prompt-injection defences filter the request
3. Crane service adds twin state, medications and schedule as context and calls Requesty (gemma-4-31b-it)
4. Reply appears with a mascot expression: caring, attentive, celebrating or alert

Key details

- Explains clinician orders exactly as recorded
- Refuses prescribing, dose changes and doubling missed doses
- Deflects non-medical questions

Where in the code

```
services/ai_assistant_service.py
services/requesty_crane_client.py
api/ai_assistant.py
components/CraneDoctorMascot.tsx
table: AIChatMessage
```

7. Medication management and dose reminders

Clinicians stay in control of prescriptions; patients get timed reminders and can confirm each dose.

How it works

1. Doctor picks the patient explicitly, then sets drug, dose, route, frequency, dates, instructions and precautions
2. Engine turns frequency into timing slots, for example Labetalol 100 mg at 08:00 and 20:00
3. Reminders sync to the patient notification center
4. Patient confirms a dose, which is saved in the dose log

Key details

- 6 monographs: labetalol, methyldopa, aspirin 81 mg, nifedipine, hydralazine, magnesium sulfate
- Warnings shown with each order

Where in the code

```
services/medication_service.py
api/medications.py
views/MedicationsView.tsx
tables: MedicationOrder, MedicationDoseLog
```

8. Doctor scheduling and patient booking

Doctors publish their availability and patients book consultations without double-booking.

How it works

1. Doctor sets weekly hours (Mon to Sun), slot length (15 to 60 min) and breaks
2. Schedule is saved to the database, with separate unavailability entries
3. Patient, or Dr. Crane on their behalf, picks an open slot
4. Booking is atomic and sends a high-priority alert to the doctor

Key details

- Slots are computed from availability minus existing bookings
- Same flow serves the patient and clinician portals

Where in the code

```
services/appointment_service.py
api/appointments.py
views/AppointmentsView.tsx
tables: DoctorAvailability, Appointment
```

9. Deterministic emergency triage

Severe warning signs are caught by fixed rules before any AI model sees the message.

How it works

1. Rules check: unresponsive severe headache, visual scotoma, RUQ or epigastric pain, dyspnea, heavy bleeding, BP at or above 160/110
2. A match returns a red emergency state telling the patient to go to emergency triage or call 911
3. A high-priority alert is dispatched to the on-call clinician (Dr. Elena Vance)
4. The event is stored and shown in the clinician portal

Key details

- Runs before the LLM, so it works even if AI is down
- Aligned with ACOG / ISSHP criteria

Where in the code

```
api/emergency.py
services/notification_service.py
table: EmergencyEvent
```

10. Welcome screen and immersive UI

A calm first impression and a consistent clinical-glass interface for both roles.

How it works

1. Welcome screen shows real maternal photography and origami crane animation
2. Delivery transition animates the move into the portal after login
3. Layout adapts navigation to the signed-in role (clinician or patient)
4. Tailwind v4 glassmorphic cards, badges and metric tiles are shared across views

Key details

- 4 bundled maternal photos
- Dr. Crane mascot is SVG with 4 expressions
- Protected routes guard clinician-only pages

Where in the code

```
views/FrontWelcomeView.tsx
components/MaternalOrigamiBackground.tsx
components/CraneDeliveryTransition.tsx
components/Layout.tsx
context/AuthContext.tsx
```

11. Devices and ambulatory measurements

Brings home readings into the twin alongside clinic data.

How it works

1. Device API registers connections: BLE sphygmomanometer, SpO2, scale and CGM
2. Connector layer adapts each device type into a common measurement format
3. Readings are stored as DeviceMeasurement rows flagged as patient-reported
4. New readings update twin state and feed the risk model

Key details

- Evidence label: patient-reported telemetry
- Sensor noise can be stress-tested in the simulation lab

Where in the code

```
services/device_connectors.py
api/devices.py
api/measurements.py
views/DevicesAndMeasurementsView.tsx
```

12. Security, audit and notifications

Cross-cutting services that keep every role isolated and every action traceable.

How it works

1. Passwords hashed with PBKDF2-HMAC-SHA256; 24-hour JWT bearer tokens
2. Role-based access (clinician, patient, admin): wrong role gets 403, other patients return 404 so IDs cannot be probed
3. audit_service records review decisions and key actions, shown in Audit history
4. notification_service delivers dose, appointment and emergency alerts to the header center

Key details

- Production refuses weak SECRET_KEY (under 32 chars)
- CORS origins configurable
- Demo mode seeds synthetic data at startup

Where in the code

```
core/security.py · core/config.py
api/deps.py · api/auth.py
api/audit.py
services/audit_service.py
services/notification_service.py
```

Clinical workspace: the remaining views

Dashboard
Risk, vitals, twin and alerts at a glance

Pregnancy timeline
Longitudinal course by gestational week

Observations
Vitals and labs with evidence-level labels

Medical records
Uploaded PDFs, report summaries, detail view

Clinician review queue
Pending AI-extracted fields awaiting approval

Audit history
Clinician-only log of reviews and actions

Settings
Profile, system and Ollama status

Notification center
Dose, booking and emergency alerts in the header

Folder structure (1/2): backend and repo root

backend/ · core, data, tests

```
backend/
├── .env.example · requirements.txt
├── storage/documents/   demo PDF
├── tests/               72 tests
│   ├── test_ai_assistant.py
│   ├── test_backend.py
│   ├── test_devices_measurements.py
│   ├── test_medications_appointments.py
│   ├── test_requesty_crane.py
│   └── test_simulation_lab.py
└── app/
    ├── main.py       FastAPI, 17 routers
    ├── core/
    │   ├── config.py     env, CORS, keys
    │   ├── database.py   engine, Base
    │   └── security.py   JWT, PBKDF2
    ├── models/entities.py  21 tables
    ├── schemas/models.py   Pydantic
    └── seed/seed_data.py   synthetic
```

backend/app/api · 17 routers

```
app/api/
├── ai_assistant.py   Dr. Crane chat
├── appointments.py   booking
├── audit.py          audit trail
├── auth.py           login, JWT
├── camera_research.py
├── clinician.py      review queue
├── deps.py           auth deps
├── devices.py        IoT links
├── documents.py      PDF review
├── emergency.py      red-flag triage
├── measurements.py   device data
├── medications.py    orders, doses
├── notifications.py
├── observations.py   labs, vitals
├── patients.py
├── predictions.py    14-day risk
├── simulation.py     what-if lab
└── twin.py           twin state
```

services/ and repo root

```
app/services/
├── ai_assistant_service.py
├── ai_extractor.py      Ollama OCR
├── appointment_service.py
├── audit_service.py
├── device_connectors.py
├── document_processor.py
├── medication_service.py
├── notification_service.py
├── prediction_service.py
├── requesty_crane_client.py
├── simulation_service.py
└── twin_service.py

health-twin/ (repo root)
├── backend/  frontend/
├── README.md · LICENSE (MIT)
├── pytest.ini · test_api.py
├── run_backend.bat · run_frontend.bat
├── start_all.ps1 · .gitignore
└── PREGTWIN_*_REPORT.md  x5
```

Folder structure (2/2): React + TypeScript frontend

frontend/ · config, public, src core

```
frontend/
├── index.html · package.json
├── vite.config.ts · .oxlintrc.json
├── tsconfig.json / .app / .node
├── public/
│   ├── favicon.svg · icons.svg
│   ├── images/maternal/  4 photos
│   ├── models/
│   │   ├── face_landmarker.task
│   │   └── maternal-body.glb
│   └── wasm/  MediaPipe x6 files
└── src/
    ├── main.tsx · App.tsx  routes
    ├── App.css · index.css
    ├── api/client.ts  REST client
    ├── assets/  hero.png, SVGs
    ├── context/
    │   ├── AuthContext.tsx
    │   └── TransitionContext.tsx
    └── tests/
        ├── ai_doctor_system.test.tsx
        └── frontend.test.tsx
```

src/views · 18 pages

```
src/views/
├── FrontWelcomeView.tsx  landing
├── LoginView.tsx
├── DashboardView.tsx
├── MaternalTwinView.tsx  3D twin
├── PregnancyTimelineView.tsx
├── RiskForecastView.tsx
├── SimulationLabView.tsx
├── CameraSignalResearchView.tsx
├── DevicesAndMeasurementsView.tsx
├── ObservationsView.tsx
├── MedicalRecordsView.tsx
├── DocumentDetailView.tsx
├── ClinicianReviewQueueView.tsx
├── AuditHistoryView.tsx
├── MedicationsView.tsx
├── AppointmentsView.tsx
├── AIAssistantView.tsx  Dr. Crane
└── SettingsView.tsx
```

src/components and src/services

```
src/components/
├── Layout.tsx
├── NotificationCenter.tsx
├── MaternalTwin3D.tsx  Three.js
├── CraneDoctorMascot.tsx  SVG
├── CraneDeliveryTransition.tsx
├── MaternalOrigamiBackground.tsx
├── MaternalOrigamiCompanion.tsx
└── ui/
    ├── Alert · Badge · Button
    └── Card · MetricCard · Modal
        PageHeader  (.tsx)

src/services/
├── faceLandmarkerService.ts
│     MediaPipe 478 landmarks
├── rppgProcessor.ts
│     CHROM pulse estimation
└── stressAnalysisPipeline.ts
      multimodal feature vector
```

Backend deep dive: 17 routers, 12 services, 21 tables

Identity & access

auth · deps · patients · audit

Clinical data

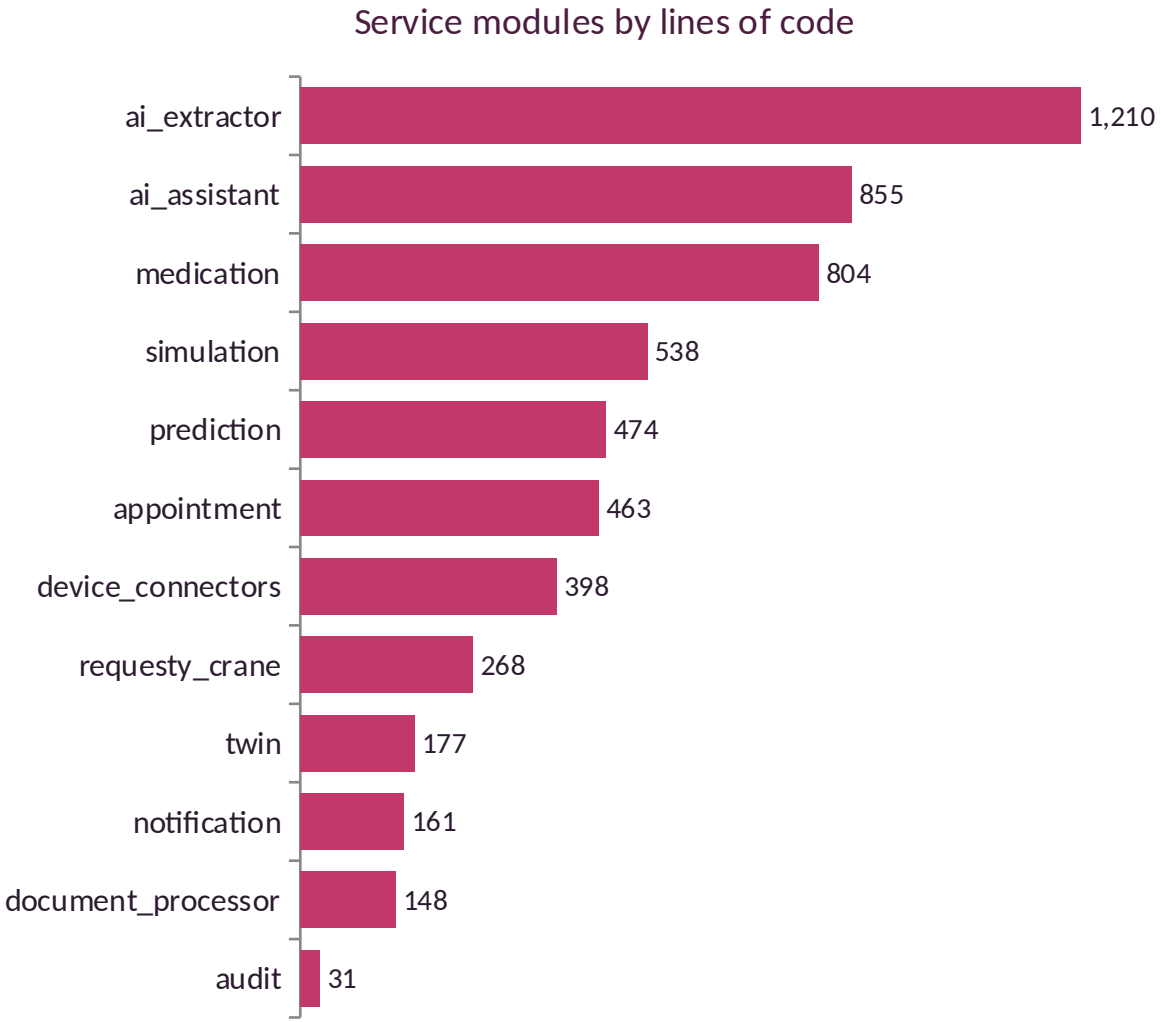
documents · observations · devices · measurements

Intelligence

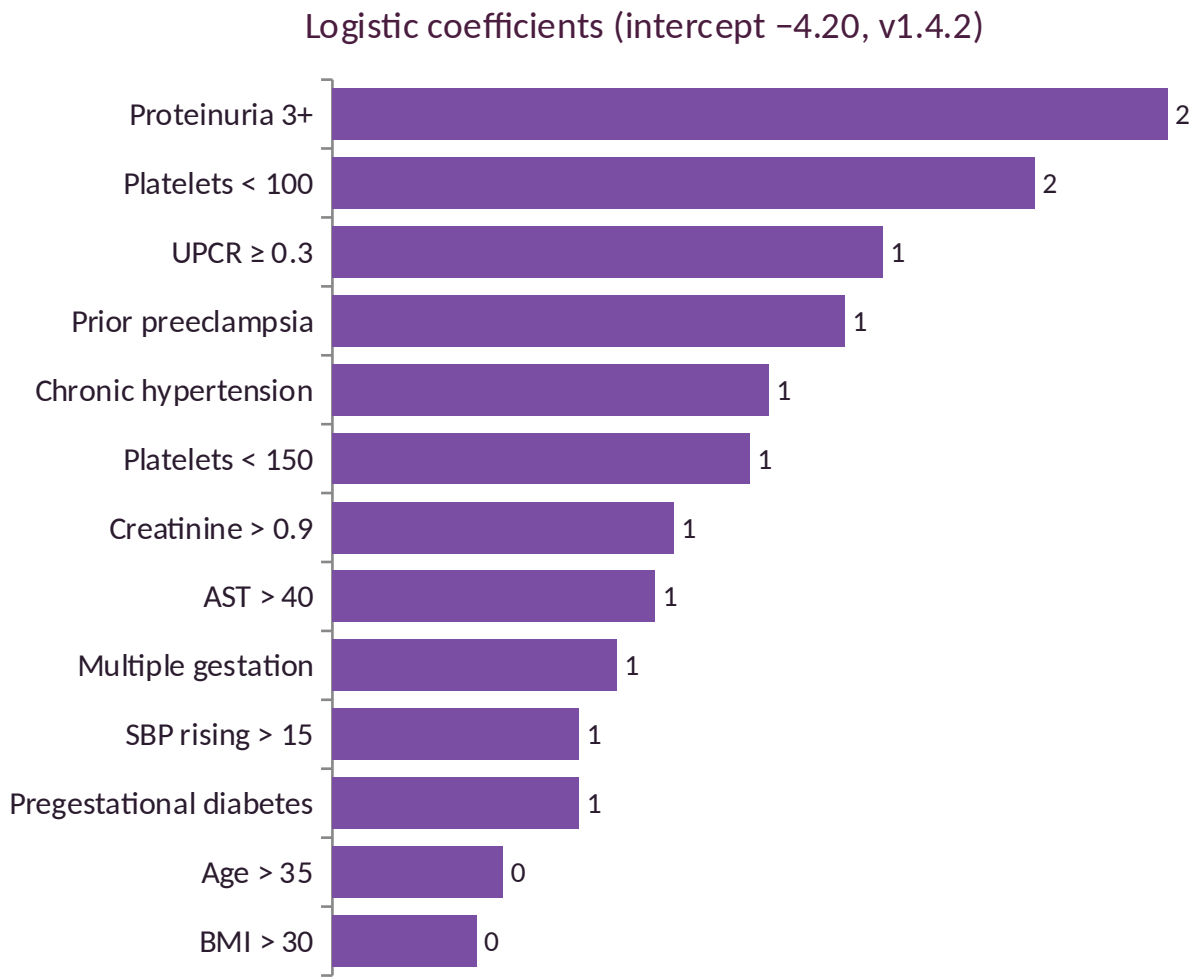
twin · predictions · simulation · camera_research

Care & safety

medications · appointments · notifications · emergency · clinician · ai_assistant



Risk model: calibrated 14-day preeclampsia forecast



Plus continuous terms per mmHg above baseline: SBP 0.048, DBP 0.052, MAP 0.035

≥ 20.0 week gate

Earlier pregnancies return INSUFFICIENT_DATA per ACOG / ISSHP.

Zero temporal leakage

Features use only observations at or before prediction time.

Calibrated and explainable

Platt-calibrated probability, 95% CI, SHAP-style contributions.

Honest evaluation

Reports not_evaluated rather than fabricating AUROC scores.

Synthetic demo cohort and verification record

72
backend pytest tests passing

25
frontend Vitest tests passing

8
audit defects fixed (PT-001 to PT-008)

0
real patient records: all data synthetic

Dr. Elena Vance, MD
Maternal-fetal medicine. Prescribes, reviews queue, receives emergency alerts.

Maya Lin · 31y · 31.4 wk
High risk: SBP 122 → 142, 1+ proteinuria, UPCR 0.34. Labetalol, methyldopa, aspirin.

Sophia Reyes · 36y · 28 wk
Moderate risk: chronic hypertension, prior preeclampsia, stable monitoring.

Clara Novak · 26y · 16 wk
Early guard: shows INSUFFICIENT_DATA below 20 weeks.

Audit fixes include accept-all preserving clinician corrections, excluding rejected fields, and context-aware glucose and gestational-age extraction.

Run it locally in five steps

1

Install Ollama

ollama pull llama3.2:3b (~2 GB)

2

Clone repo

git clone vatsalpro/health-twin

3

Start backend

venv, pip install, seed_data, uvicorn on :8008

4

Start frontend

npm install, npm run dev

5

Open the app

localhost:5173 · API docs at :8008/docs

Research prototype, not a diagnostic device

PregTwin is clinical decision support. All patient data is synthetic, camera analysis never leaves the browser, and Ollama keeps clinical text on-device. MIT licensed.

Every beat matters. Every mother matters. · *Vatsal Kumar, Team kvatsal03*